LIBSPACE

Project Title

Libspace API - A Robust Backend for Library Operations.

Team Leader: K.Madhumitha- madhum8329@gmail.com

Team members: S.Leenadevi- leenadevisugumaran@gmail.com

N.Lokeshwari- lokeshwari02122006@gmail.com

V.Mageshwari- magimageshwari354@gmail.com

1. Project Overview

LibSpace is a micro web-based library management system that provides secure user authentication and basic book management features. The application allows users to register, log in, log out, and delete their accounts. Once authenticated, users can perform CRUD (Create, Read, Update, Delete) operations on books, including adding new books, viewing the list of books, updating book details, and deleting books.

The main goal of LibSpace is to demonstrate authentication flow and CRUD operations while backend, and database functionalities in a structured manner.

2. Project Instructions – *LibSpace*

1. Register a new user account using the registration form.
2. Log in with the registered credentials to access the dashboard.
3. After successful login, perform book management operations:

    - Add a new book.
    - View the list of available books.
    - Get book by ID
    - Update book details.
    - Delete a book.

4. Users can log out securely after completing operations.
5. Users also have the option to delete their account permanently from the system.

These instructions describe how the system should be used once it is running.

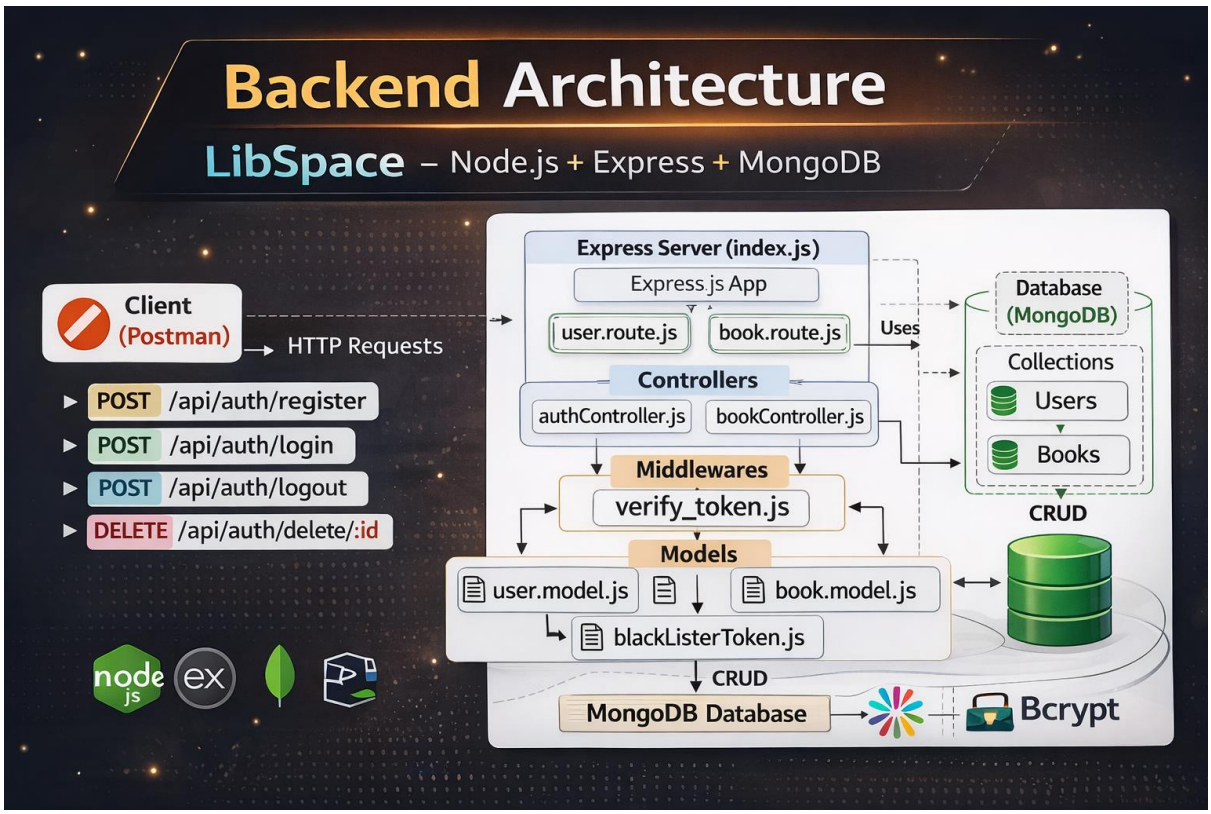
Pre-requisites

Before running or developing LibSpace, ensure the following:

- Basic knowledge of:
  - JavaScript
  - Backend concepts (e.g., Node.js / Express)
  - Database fundamentals (CRUD operations)
- Software Requirements:
  - Node.js and npm installed
  - Database setup (MongoDB / MySQL)
  - Code editor (e.g., VS Code)
  - Web browser (Chrome/Firefox)

- Basic understanding of:
  - REST APIs
  - Authentication (sessions or JWT)
  - Environment variables configuration

3. Project Architecture



Technical Architecture

LibSpace follows a **Layered Backend Architecture** consisting of:

1. **API Layer (Routes)**
2. **Business Logic Layer (Controllers)**
3. **Data Access Layer (Models / Database)**

This structure ensures clean code organization, scalability, and maintainability.

1. API Layer (Routes):

The API layer handles incoming HTTP requests and defines application endpoints.

- Built using: Node.js + Express.js
- Responsible for:
    - Defining routes (e.g., `/register`, `/login`, `/books`)
    - Mapping routes to controllers
    - Handling HTTP methods (GET, POST, PUT, DELETE)
    - Applying middleware (authentication, validation)

Example Endpoints:

- POST → Register User
- POST → Login User
- DELETE → Delete User
- POST → Add Book
- GET → Fetch Books
- PUT → Update Book
- DELETE → Delete Book

2. Business Logic Layer (Controllers):

The controller layer contains the core application logic.

Responsibilities:

- Processing user registration and login
- Password hashing (e.g., bcrypt)
- Token generation (JWT) / session handling
- Implementing CRUD operations for books
- Validating request data
- Handling errors and sending responses

This layer acts as a bridge between routes and database.

3. Data Access Layer (Models / Database):

The data layer manages persistent storage.

- Database: MongoDB / MySQL
- Models define schema structure for:
    - Users (username, email, password)
    - Books (title, author, published_date, etc.)

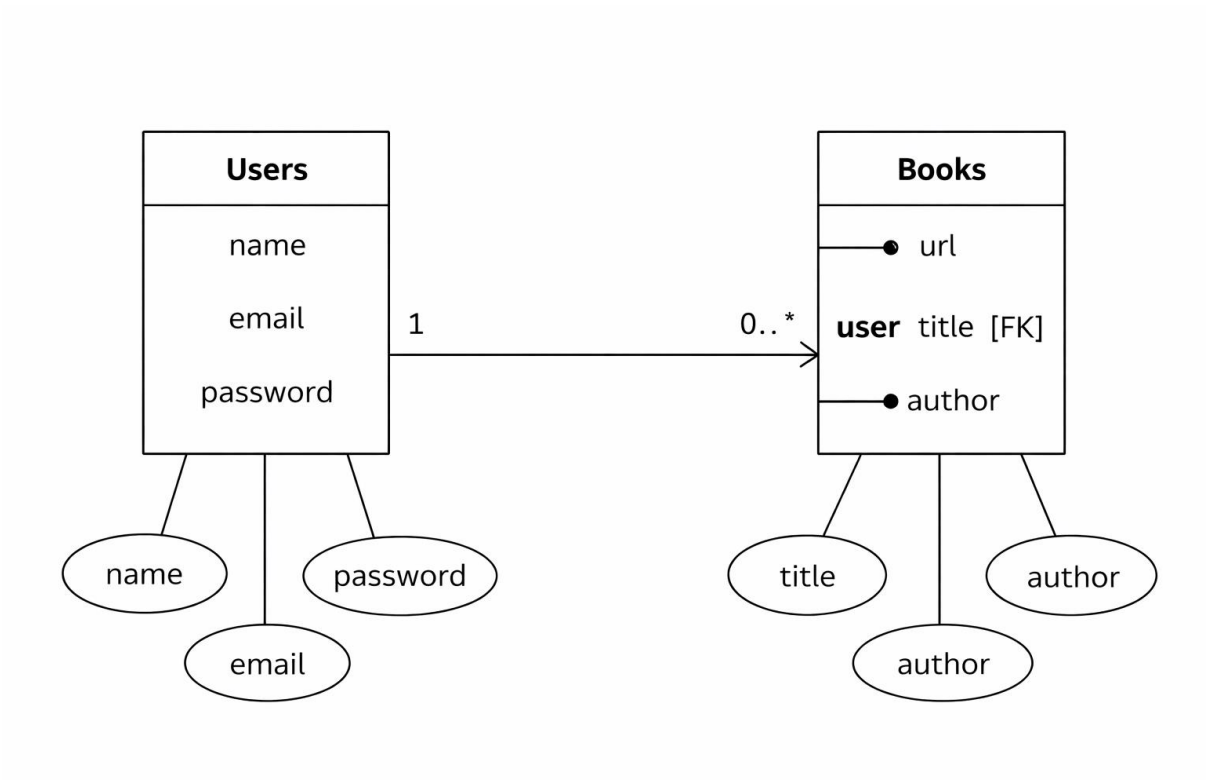
Responsibilities:

- Executing database queries
- Storing and retrieving data
- Maintaining data integrity

4. Request Flow:

1. Client sends HTTP request to API endpoint.
2. Route receives the request and forwards it to the controller.
3. Controller processes business logic.
4. Controller interacts with the database via models.
5. Database returns data to controller.
6. Controller sends structured JSON response back to client.

ER Diagram:



Features:

1. User Authentication

- User Registration (name, email, password)
- Secure password storage (hashed password)
- User Login with credential verification
- Token-based authentication (JWT)
- Delete User Account
- Automatic timestamps (createdAt, updatedAt)

2. Book Management (CRUD Operations)

Each book is linked to a specific user using a reference (`user: ObjectId`).

- Add a new book (url, title, author)
- Get all books of a specific user
- Get book by ID
- Update book details
- Delete a book
- Books are protected and accessible only to the authenticated user

3. User–Book Relationship

- One-to-Many Relationship:
  - One User → Can have multiple Books
  - Each Book → Belongs to one User (via ObjectId reference)
- Maintains data integrity using MongoDB references

4. Security & Validation

- Input validation for required fields
- Protected routes using authentication middleware
- Error handling for invalid requests
- Unauthorized access prevention

5. RESTful API Design

- Follows REST principles
- Uses standard HTTP methods:
  - POST
  - GET
  - PUT
  - DELETE
- Returns structured JSON responses

Roles and Responsibilities:

Requirement Analysis

- Understanding project requirements (authentication + book CRUD).
- Designing database schema for Users and Books.
- Defining relationship between User and Book (One-to-Many).

Database Design

- Creating Mongoose schemas:
    - User Schema (name, email, password, timestamps)
    - Book Schema (url, title, author, user reference)
- Establishing ObjectId reference between Book and User.
- Ensuring data consistency and integrity.

API Development

- Developing RESTful API endpoints.
- Implementing:
    - User Registration
    - User Login
    - Delete User
    - Add Book
    - Get Books
    - Update Book
    - Delete Book

- Structuring routes and controllers properly.

Authentication & Security

- Implementing password hashing (bcrypt).
- Generating JWT tokens for authentication.
- Protecting private routes using middleware.
- Handling unauthorized access and validation errors.

Error Handling & Validation

- Validating required fields.
- Handling invalid credentials.
- Managing database errors.
- Returning structured JSON responses.

Testing & Debugging

- Testing APIs using Postman.
- Verifying CRUD operations.
- Fixing bugs and ensuring proper response status codes.

User Flow – *LibSpace Backend*

The user flow describes how a user interacts with the system APIs step-by-step.

---

1 User Registration

- User sends a **POST request** to `/api/auth/register`
- System validates input fields (name, email, password)
- Password is hashed using bcrypt
- User data is stored in the database
- Success response is returned

---

2 User Login

- User sends a **POST request** to `/api/auth/login`
- System verifies email and password
- If valid, a **JWT token** is generated
- Token is sent back in the response
- User is now authenticated

---

3 Access Protected Routes

User sends requests with:

```
Authorization: Bearer <token>
```

-
- Middleware verifies JWT
- If valid → request proceeds
- If invalid → access denied

---

4 Book Operations (After Login)

Authenticated user can:

- ➕Add Book → POST `/api/book/add`
- Get Books → GET `/api/book`
- ✏️Update Book → PUT `/api/book/:id`
- ❌Delete Book → DELETE `/api/book/:id`

Each book is linked to the logged-in user.

---

5 Logout

- User sends request to `/api/auth/logout`
- Token is stored in blacklist
- User session becomes invalid

---

6 Delete User (Optional)

- User sends DELETE request with userId
- Account is removed from database

Overall Flow Summary

Register → Login → Receive Token → Access Protected Routes → Perform CRUD → Logout

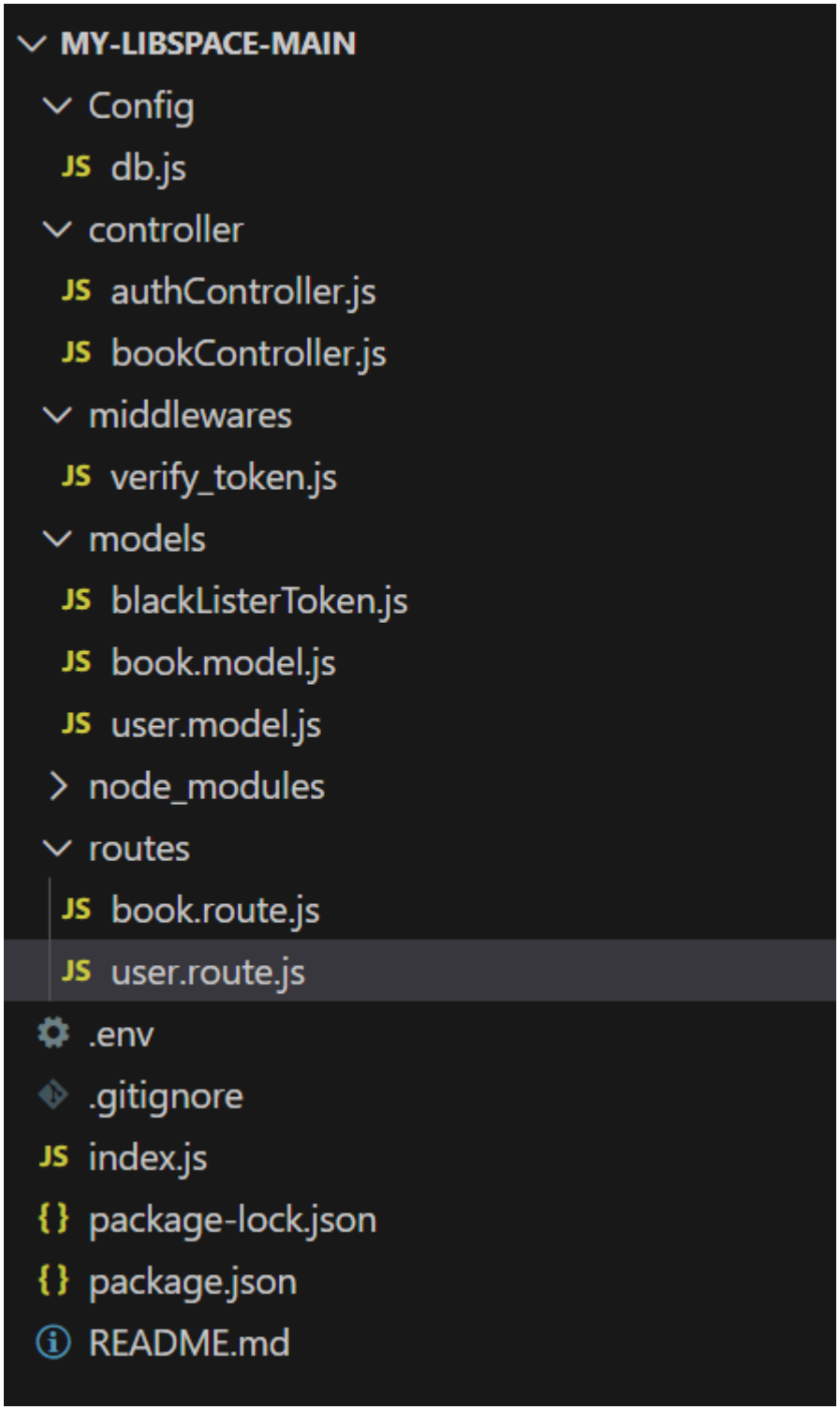
```
Authorization: Bearer <token>
```

Project Setup and Configuration:

Folder Structure:

```
MY-LIBSPACE-MAIN/
│
├── Config/
│   └── db.js
│
├── controller/
│   ├── authController.js
│   └── bookController.js
│
├── middlewares/
│   └── verify_token.js
│
├── models/
│   ├── blackListerToken.js
│   ├── book.model.js
│   └── user.model.js
│
├── routes/
│   ├── book.route.js
│   └── user.route.js
│
```

```
├── node_modules/
│
├── .gitignore
├── index.js
├── package.json
├── package-lock.json
└── README.md
```



Project setup:

1. Install Dependencies

Install all required npm packages:

```
npm install
```

2. Configure Environment Variables

Create a `.env` file in the root directory and add the following:

```
PORT=5000

MONGO_URI=your_database_connection_string

JWT_SECRET=your_secret_key
```

- Replace `your_database_connection_string` with your MongoDB/MySQL connection string.
- Replace `your_secret_key` with any secure random string.

3. Start the Database

- Make sure MongoDB/MySQL is installed and running on your system.
- Verify the connection string matches your database configuration.

4. Run the Server

Start the application using:

```
npm start
```

Or (if using nodemon):

```
npm run dev
```

6. Access the Application

Open your browser and navigate to:

http://localhost:5000

The LibSpace application should now be running successfully.

Backend Development

The backend of LibSpace is developed using **Node.js and Express.js**, following RESTful architecture principles. It handles authentication, business logic, database operations, and API responses. The backend ensures secure data handling, structured routing, and efficient communication with MongoDB.

---

1 API Development & Routing

The backend APIs are built using Express.js. Routes are structured into separate files for user and book operations to maintain modularity.

- User Routes handle:
    - Registration
    - Login
    - Logout
    - Delete User

- Book Routes handle:
    - Add Book
    - Get Books
    - Update Book
    - Delete Book

Each route forwards requests to corresponding controllers where business logic is implemented.

---

2 Database Integration & Authentication

MongoDB is used as the database, integrated using Mongoose for schema modeling and data management.

- User passwords are securely hashed using bcrypt.
- JWT (JSON Web Token) is generated during login for authentication.
- Protected routes use middleware to verify tokens.
- A blacklist collection is maintained to invalidate tokens during logout.

This ensures secure access control and efficient data management within the backend system.

DATABASE DEVELOPMENT:

The database for **LibSpace** is developed using **MongoDB**, a NoSQL document-based database.

The application stores two main collections:

1. **Users**

2. **Books**

A one-to-many relationship is maintained:

- One User → Can have multiple Books
- Each Book → Belongs to one User (via ObjectId reference)

MongoDB stores data in **JSON-like documents**, making it flexible and scalable.

CONFIGURE MongoDB:

MongoDB is configured using **Mongoose**, an ODM (Object Data Modeling) library for Node.js.

Steps:

- Install mongoose:

```
npm install mongoose
```

- Create a database connection file inside `Config/db.js`
- Connect using MongoDB URI

Environment variable (.env):

```
MONGO_URI=mongodb://127.0.0.1:27017/libspace
```

CREATE DATABASE CONNECTION

File: `Config/db.js`

```
const mongoose = require("mongoose");

const connection = mongoose.connect(process.env.MONGO_URI);

module.exports = {

  connection,
```

```
};
```

In index.js:

```
const { connectionDB } = require("./Config/db");


app.listen(5000, async () => {
  try {
    await connectionDB;
    console.log("Connected to Database");
  } catch (err) {
    console.log("Database connection failed");
  }
});
```

This ensures the server starts only after a successful database connection.

CREATE SCHEMA AND MODELS:

Schemas define the structure of documents inside MongoDB collections.

User Schema

File: models/user.model.js

```
const mongoose = require("mongoose");


const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true },
```

```
    email: { type: String, required: true },
    password: { type: String, required: true },
  },
  { versionKey: false, timestamps: true }
);

const UserModel = mongoose.model("User", userSchema);

module.exports = UserModel;
```

Book Schema:

File: `models/book.model.js`

```
const mongoose = require("mongoose");

const bookSchema = new mongoose.Schema({
  url: { type: String, required: true },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
  title: { type: String, required: true },
```

```
  author: { type: String, required: true },

});



const BookModel = mongoose.model("Book", bookSchema);



module.exports = BookModel;
```

Project Execution:

Project execution steps:

1.Install Dependencies

Open terminal inside the project folder and run:

```
npm install
```

This installs all required packages from package.json.

2**. Start the Server**

Run the following command:

npm start

OR (if using nodemon):

npm run dev

4. Database Connection

When the server starts:

- connectDb() connects to MongoDB.
- If successful, the console displays:

Connected to Database

Our server is working at PORT :5000

5.**Test API Endpoints**

Use Postman or any API testing tool to test the endpoints:

□ User Routes

- POST → /api/auth/register
- POST → /api/auth/login
- DELETE → /api/auth/delete

□ Book Routes

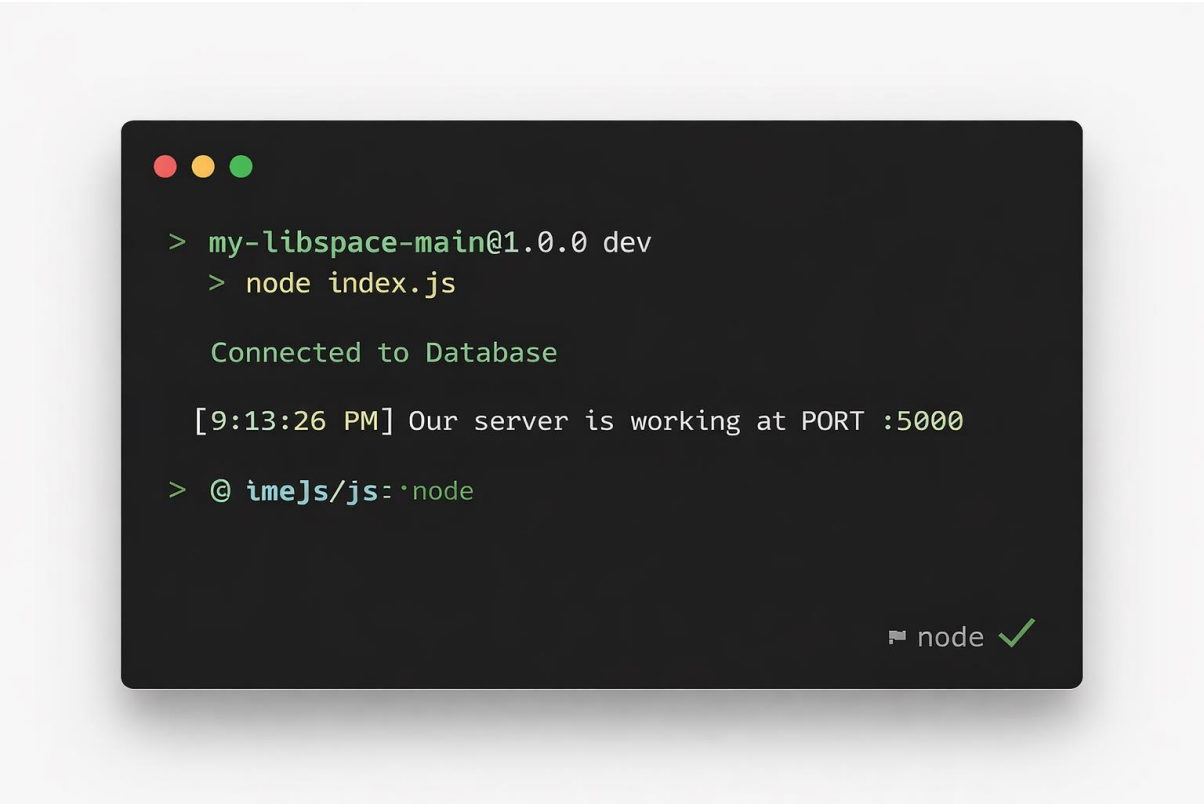
- POST → /api/book/add
- GET → /api/book/
- PUT → /api/book/:id
- DELETE → /api/book/:id

Base URL:

http://localhost:5000

Output Screenshots:

Output on terminal once server starts:



User registration output:

libspace / **userRegister**

POST http://localhost:5000/api/authuser/register

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "name":"Sri Chandana",
    "email": "chandana3@gmail.com",
    "password": "123456"

}
```

Body Cookies Headers (7) Test Results 200 OK 851 ms 293 B

{} JSON Preview Visualize

```
{
    "success": true,
    "message": "User registered successfully!"
}
```

User Login output:

POST http://localhost:5000/api/authuser/login

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "email": "chandana3@gmail.com",
    "password": "123456"
}
```

Body Cookies Headers (7) Test Results 200 OK 209 ms 739 B

{} JSON Preview Visualize


```
{
    "msg": "Logged In Successfully",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJpZCI6IjY5OTZiMDBhODZmMWE3MzQyYjlkMDYwNSIsImVtYWlsIjoiY2hhbmRhbmEzQGdtYWlsLmN
        3MTQ4MzQxMiwiZXhwIjoxNzcxNDg3MDEyfQ.MUTwlsM7iGGeZyEZmU3chsMBfOJhLHlpKCaccdRTZQs
    "user": [
        {
            "_id": "6996b00a86f1a7342b9d0605",
            "name": "Sri Chandana",
            "email": "chandana3@gmail.com",
            "password": "$2b$10$9OBW1X/3Q/.kqmjyFCLAoOq/Q2Z9RHg6fCQVhMZf80..zSwkVDydm",
            "createdAt": "2026-02-19T06:39:07.014Z",
```


Adding book:

libspace / **addBook**

POST http://localhost:5000/api/book/addbook

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "url":"www.libspacebook.com",
    "title": "think like a monk",
    "author": "jay shetty"
}
```

Body Cookies Headers (7) Test Results 200 OK 83 ms 268 B

JSON Preview Visualize

```
{
    "msg": "Book added successfully"
}
```

Get All books:

libspace / **getallBooks**

GET http://localhost:5000/api/book/getBooks

Docs Params Authorization Headers (7) Body Scripts Settings

Auth Type

Be...

Token

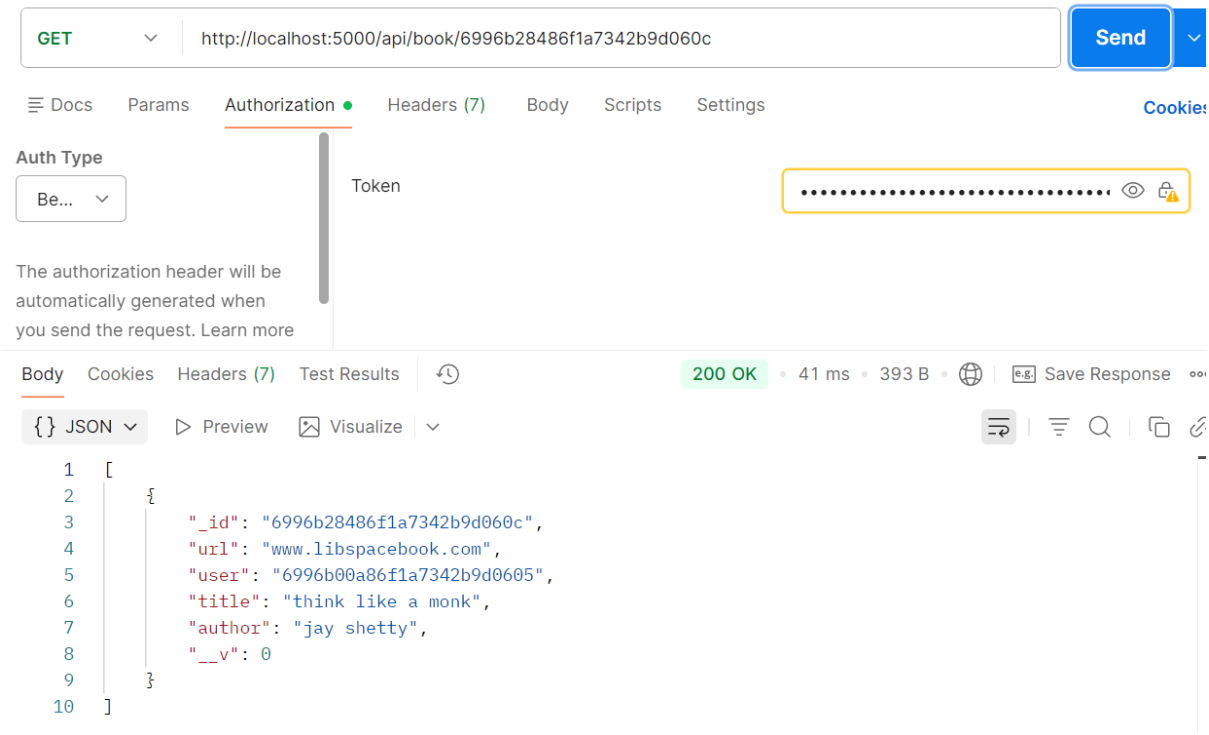
The authorization header will be automatically generated when you send the request. Learn more

Body Cookies Headers (7) Test Results 200 OK 38 ms 393 B

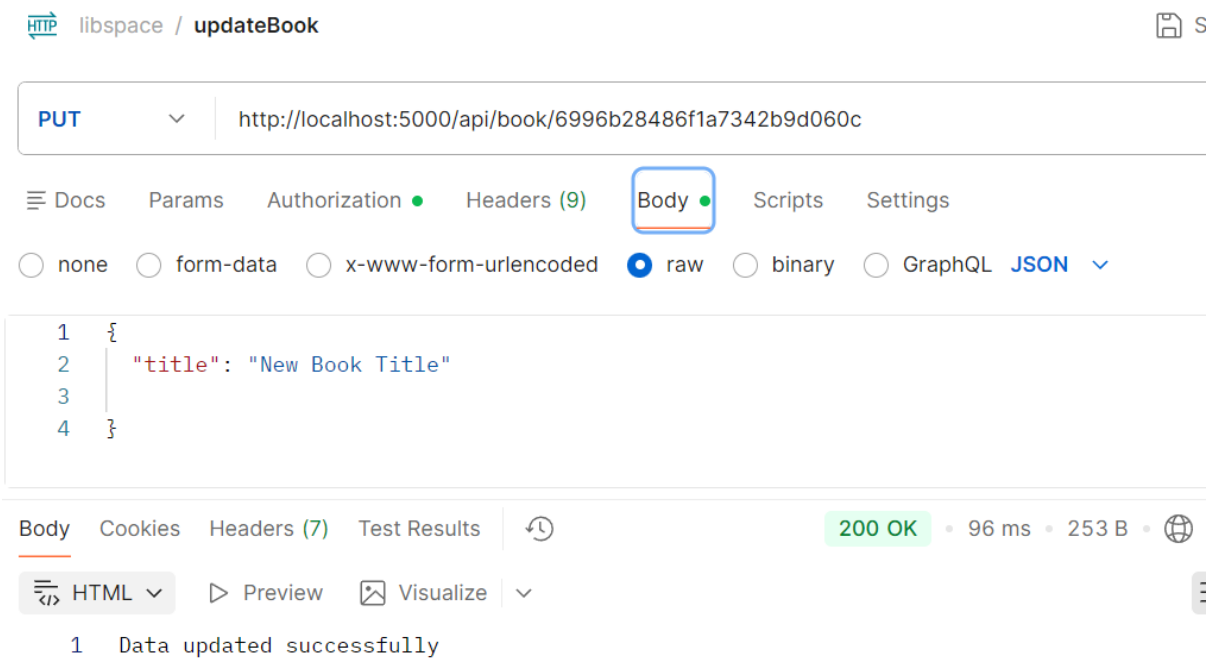
JSON Preview Visualize

```
[
    {
        "_id": "6996b28486f1a7342b9d060c",
        "url": "www.libspacebook.com",
        "user": "6996b00a86f1a7342b9d0605",
        "title": "think like a monk",
        "author": "jay shetty",
        "__v": 0
    }
]
```

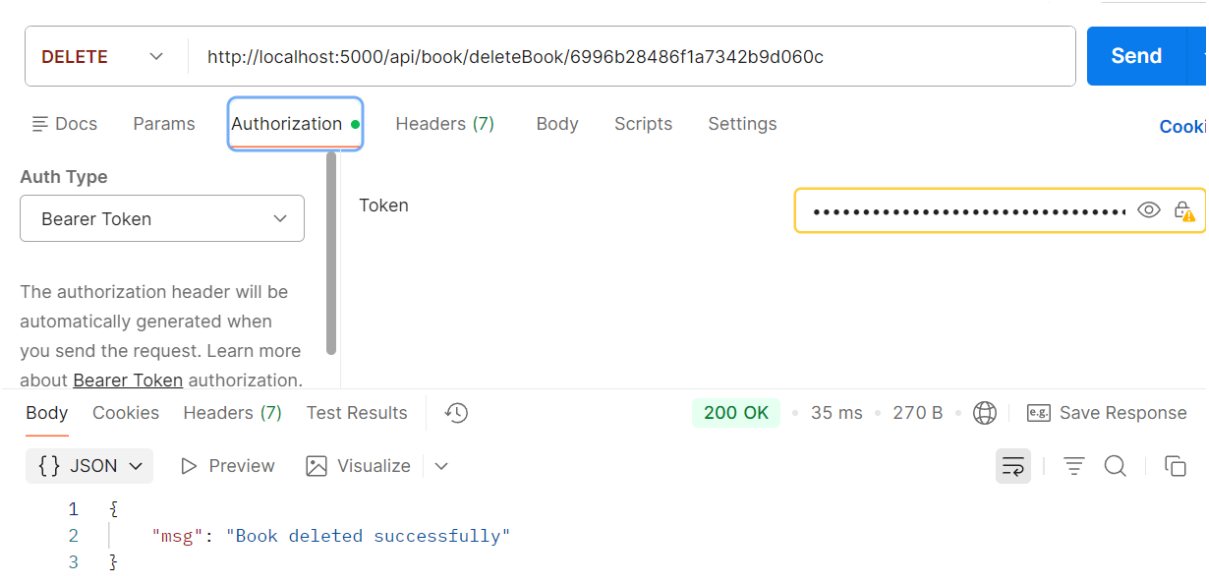
Get book by ID:



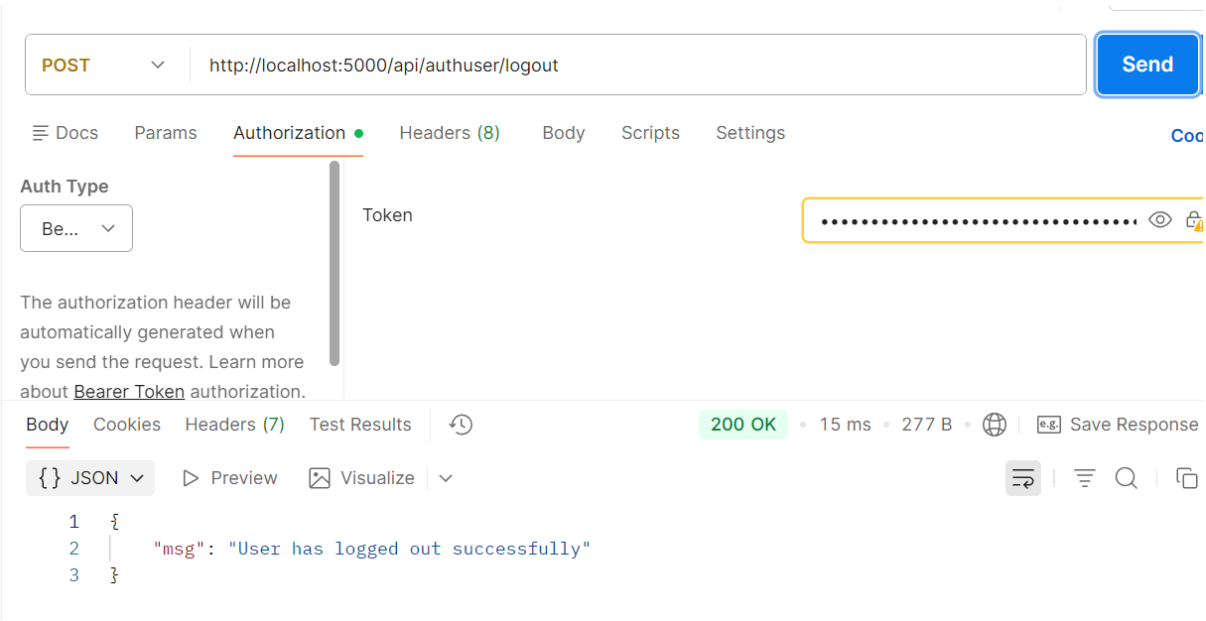
Update book:



Delete book:



User Logout:



Demo video:

[LibSpaceDemo]

Drive Link : [Libspace code]